

```

for j in range(0,len(triangles)):
    commonpoints = 0
    for k in range(0,len(triangles[i])):
        for n in range(0,len(triangles[j])):
            if triangles[i][k] == triangles[j][n]:
                commonpoints += 1
    if commonpoints == 2:
        lines.append([list(centers[i][0]),list(centers[j][0])])

lc = mc.LineCollection(lines, linewidths = 1)

ax.add_collection(lc)

pl.savefig(str(thename) + '.png')
pl.close()

```

***Listing 7-4:** A function that plots triangles, circumcenters, circumcircles, Voronoi points, and Voronoi boundaries*

First, we add new arguments that specify exactly what we want to plot. Remember that in this chapter we have worked with points, edges, triangles, circumcircles, circumcenters, DTs, and Voronoi boundaries. It could be overwhelming to the eye to plot all of these together, so we will add `plotcircles` to specify whether we want to plot our circumcircles, `plotpoints` to specify whether we want to plot our collection of points, `plottriangles` to specify whether we want to plot our DT, `plotvoronoi` to specify whether we want to plot our Voronoi diagram edges, and `plotvpoints` which to specify whether we want to plot our circumcenters (which are the vertices of the Voronoi diagram edges). The new additions are shown in bold. One addition plots the Voronoi vertices (circumcenters), if we have specified in our arguments that we want to plot them **❶**. The longer addition plots the Voronoi edges **❷**. We’ve also specified a few `if` statements that allow us to plot, or not plot, triangles, vertices, and circumcircles, as we prefer.

We’re almost ready to call this plotting function and see our final Voronoi diagram. However, first we need to get the circumcenters of every triangle in our DT. Luckily, this is very easy. We can create an empty list called `circumcenters` and append the circumcenter of every triangle in our DT to that list, as follows: